Dynamic Programming & Recursion

Complete Cheatsheet

Core Concepts

What is Dynamic Programming?

DP = Recursion + Memoization

- Break problem into smaller subproblems
- Solve each subproblem once
- Store results (memoization)
- Reuse stored results

When to use DP?

✓ Optimal substructure
✓ Overlapping subproblems
✓ Can define recurrence relation

Two Approaches

1. Top-Down (Memoization)
→ Start from problem
→ Recursively break down
→ Cache results as you go
→ Uses recursion + memo array

2. Bottom-Up (Tabulation)
→ Start from base cases
→ Build up iteratively
→ Fill DP table in order
→ Uses loops + DP array

Recursion Fundamentals

Three Components:

1. **Base case** - When to stop
2. **Recursive case** - Self-call with smaller input
3. **Progress** - Move toward base case

Call Stack:

- Each call creates stack frame
- Frames pause until child returns
- Answers bubble back up

Problem 1: Climbing Stairs

Problem Statement

Given: n stairs, can climb 1 or 2 steps
Find: Number of distinct ways to reach top

Example: $n = 3$
Ways: $\{1+1+1, 1+2, 2+1\}$ → Answer: 3

Approach 1: Pure Recursion

Intuition

To reach step n: came from $n-1$ or $n-2$
$f(n) = f(n-1) + f(n-2)$ (Fibonacci!)

```java
int climbStairs(int n) {
    if (n == 0 || n == 1) return 1;
    return climbStairs(n-1) + climbStairs(n-2);
}
```

Complexity

Time: $O(2^n)$ - exponential tree
Space: $O(n)$ - recursion stack
Issue: Recalculates same values!

Approach 2: Memoization (Top-Down)

```java
int climbStairs(int n) {
    int[] memo = new int[n + 1];
    return helper(n, memo);
}

int helper(int n, int[] memo) {
    if (n == 0 || n == 1) return 1;
    if (memo[n] != 0) return memo[n];

    memo[n] = helper(n-1, memo) + helper(n-2, memo);
    return memo[n];
}
```

Complexity

Time: $O(n)$ - each subproblem solved once
Space: $O(n)$ - memo array + stack

Approach 3: Bottom-Up (Iterative)

```java
int climbStairs(int n) {
    if (n == 0 || n == 1) return 1;

    int[] dp = new int[n + 1];
    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}
```

Complexity

Time: $O(n)$ - single loop
Space: $O(n)$ - DP array only
Advantage: No recursion overhead

Approach 4: Space-Optimized

```java
int climbStairs(int n) {
    if (n == 0 || n == 1) return 1;

    int prev2 = 1; // dp[i-2]
    int prev1 = 1; // dp[i-1]

    for (int i = 2; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Complexity

Time: $O(n)$
Space: $O(1)$ - only 3 variables!
Key: Only need last 2 values

Common Mistakes

× Creating memo inside recursive function
✓ Create memo once, pass to all calls
× Base case: `if (n==0) return 0`
✓ Base case: `if (n==0) return 1` (1 way)
× Confusing input (stairs) with output (ways)

Problem 2: House Robber

Problem Statement

Given: Array of house values
Constraint: Can't rob adjacent houses
Find: Maximum money can rob

Example: `[2,7,9,3,1]`
Rob houses 0,2,4: $2+9+1 = 12$ → Answer: 12

Key Insight

At each house: **ROB** or **SKIP**
- Rob: get value + skip next
- Skip: consider next house

$f(i) = \max(\text{rob}, \text{skip})$

Approach 1: Recursion (Top-Down)

```java
int rob(int[] nums) {
    int[] memo = new int[nums.length];
    Arrays.fill(memo, -1);
    return robFrom(0, nums, memo);
}

int robFrom(int i, int[] nums, int[] memo) {
    if (i >= nums.length) return 0;
    if (memo[i] != -1) return memo[i];

    // Rob this house or skip it
    int rob = nums[i] + robFrom(i+2, nums, memo);
    int skip = robFrom(i+1, nums, memo);

    memo[i] = Math.max(rob, skip);
    return memo[i];
}
```

Complexity

Time: $O(n)$ - each house considered once
Space: $O(n)$ - memo + recursion stack

Approach 2: Bottom-Up (Iterative)

```java
int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int[] dp = new int[nums.length];
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        dp[i] = Math.max(
            nums[i] + dp[i-2],  // Rob this
            dp[i-1]             // Skip this
        );
    }
    return dp[nums.length - 1];
}
```

Approach 3: Space-Optimized

```java
int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int prev2 = nums[0];
    int prev1 = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        int current = Math.max(
            nums[i] + prev2,
            prev1
        );
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Space Optimization

Space: $O(1)$
Only need last 2 values: $dp[i-1]$ and $dp[i-2]$

Common Mistakes

× `robFrom(i, nums)` without memo parameter
✓ Pass memo to share across all calls
× Thinking it's fixed pattern (even/odd)
✓ It's about optimal choice at each house

Problem 3: Coin Change

Problem Statement

Given: Coin denominations, target amount
Find: Minimum coins to make amount
Return: -1 if impossible

Example: coins=`[1,2,5]`, amount=11
$5 + 5 + 1 = 11$ (3 coins) → Answer: 3

Key Insight

Try ALL coins, pick minimum:
$f(\text{amount}) = 1 + \min(f(\text{amount} - \text{coin}))$
- Can reuse same coin (unbounded)
- 1D DP (only track amount)

Approach 1: Recursion with Memoization

```java
int coinChange(int[] coins, int amount) {
    int[] memo = new int[amount + 1];
    Arrays.fill(memo, -1);
    int result = helper(coins, amount, memo);
    return result == Integer.MAX_VALUE ? -1 : result;
}

int helper(int[] coins, int amount, int[] memo) {
    if (amount == 0) return 0;
    if (amount < 0) return Integer.MAX_VALUE;
    if (memo[amount] != -1) return memo[amount];

    int min = Integer.MAX_VALUE;
    for (int coin : coins) {
        int result = helper(coins, amount - coin, memo);
        if (result != Integer.MAX_VALUE) {
            min = Math.min(min, 1 + result);
        }
    }

    memo[amount] = min;
    return min;
}
```

Complexity

Time: $O(\text{amount} \times \text{coins})$
Space: $O(\text{amount})$ - memo + stack

Approach 2: Bottom-Up (Iterative)

```java
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); // "infinity"
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (i >= coin) {
                dp[i] = Math.min(dp[i], 1 + dp[i - coin]);
            }
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}
```

Why amount+1 as infinity?

- Max coins needed = amount (all $1 coins)
- amount+1 = impossible
- Avoids overflow with MAX_VALUE

Common Mistakes

× Using -1 for impossible during recursion
✓ Use MAX_VALUE, convert to -1 at end
× Not understanding `1 +` in formula
✓ The 1 is the coin we just used!
× Confusing with 0/1 Knapsack
✓ Coin Change = unbounded (reuse allowed)

Why No Space Optimization?

Variable look-back: $dp[i - \text{coin}]$
Coin can be 1, 2, 5, 10, etc. (variable)
Must keep full array $O(\text{amount})$

Problem 4: 0/1 Knapsack

Problem Statement

Given: Items with weights & values, capacity
Constraint: Each item used 0 or 1 times
Find: Maximum value without exceeding capacity

Example:
wt=`[1,3,4]`, val=`[1,4,5]`, W=4
Take items 0,1: $1 + 4 = 5 \rightarrow$ Answer: 5

Key Insight

For each item: **TAKE** or **SKIP**
$f(i, w) = \max($
$f(i-1, w),$ // Skip
$\text{val}[i] + f(i-1, w - \text{wt}[i])$ // Take
$)$

- 2D state: (item, capacity)
- Each item used at most once (bounded)

Approach 1: Recursion with Memoization

```java
int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[][] memo = new int[n][W + 1];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dfs(W, val, wt, 0, memo);
}

int dfs(int W, int[] val, int[] wt, int i, int[][] memo) {
    if (i >= wt.length || W == 0) return 0;
    if (memo[i][W] != -1) return memo[i][W];

    int skip = dfs(W, val, wt, i + 1, memo);
    int take = 0;
    if (wt[i] <= W) {
        take = val[i] + dfs(W - wt[i], val, wt, i + 1, memo);
    }

    memo[i][W] = Math.max(skip, take);
    return memo[i][W];
}
```

Complexity

Time: $O(n \times W)$
Space: $O(n \times W)$ - 2D memo

Approach 2: Bottom-Up (2D DP)

```java
int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[][] dp = new int[n + 1][W + 1];

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            int skip = dp[i - 1][w];
            int take = 0;
            if (wt[i - 1] <= w) {
                take = val[i - 1] + dp[i - 1][w - wt[i - 1]];
            }
            dp[i][w] = Math.max(skip, take);
        }
    }
    return dp[n][W];
}
```

Index Mapping

$dp[i][w]$ = using first i items
DP row $i \rightarrow$ item at array index $i - 1$
Why? DP is 1-indexed, arrays are 0-indexed

Approach 3: Space-Optimized (1D)

```java
int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[] dp = new int[W + 1];

    for (int i = 0; i < n; i++) {
        // CRITICAL: Iterate BACKWARDS!
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = Math.max(
                dp[w],              // Skip
                val[i] + dp[w - wt[i]]    // Take
            );
        }
    }
    return dp[W];
}
```

Why Backward Iteration?

Forward ($\rightarrow$): Allows reusing same item
$\rightarrow$ Becomes Unbounded Knapsack

Backward ($\leftarrow$): Uses old values only
$\rightarrow$ Each item used at most once

✓ When updating $dp[w]$, read $dp[w - \text{wt}]$ (not yet updated)
× Forward would read already-updated value

Space Complexity

2D DP: $O(n \times W)$
1D Optimized: $O(W)$ - single array!
Only need previous row, not all rows

Code Templates

Memoization Template

```java
// Main function
ReturnType solve(params) {
    Memo[] memo = initialize;
    return helper(params, memo);
}

// Helper with memo
ReturnType helper(params, Memo memo) {
    if (base_case) return base_value;
    if (memo.has(key)) return memo.get(key);

    result = compute using helper(smaller, memo);
    memo.store(key, result);
    return result;
}
```

Bottom-Up DP Template

```java
ReturnType solve(params) {
    DP[] dp = initialize;
    dp[base] = base_value;

    for (int i = start; i <= end; i++) {
        dp[i] = compute using dp[smaller];
    }

    return dp[target];
}
```

Space Optimization Template

```java
// For fixed k-step look-back
ReturnType solve(params) {
    Type prev1 = init1;
    Type prev2 = init2;

    for (int i = start; i <= end; i++) {
        Type current = compute(prev1, prev2);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Complexity Analysis

Time Complexity Logic

Pure Recursion:
- Tree with branching factor b, depth d
- Time = $O(b^d)$ (exponential)

With Memoization:
- Each subproblem solved once
- Time = # of subproblems × work per subproblem

Examples:
Climbing Stairs: $O(n)$ - n subproblems
Coin Change: $O(\text{amount} \times \text{coins})$
Knapsack: $O(n \times W)$ - $n \times W$ subproblems

Space Complexity Logic

Memoization (Top-Down):
Space = memo size + recursion stack
- Memo: size of state space
- Stack: depth of recursion tree

Tabulation (Bottom-Up):
Space = DP table size
- No recursion stack overhead

Space Optimization:
If $dp[i]$ only depends on $dp[i-k]$ to $dp[i-1]$:
→ Can use $O(k)$ space instead of $O(n)$

Common Mistakes & Solutions

Mistake 1: Memo Scope

× **Wrong:**

```
int helper(int n) {
    int[] memo = new int[n+1]; // NEW each call!
    // ...
}
```

✓ **Correct:**

```
int solve(int n) {
    int[] memo = new int[n+1]; // Created ONCE
    return helper(n, memo);     // Shared
}
```

Mistake 2: Base Case Meaning

× **Wrong Understanding:**
"`if (n==0) return 0`" means 0 steps

✓ **Correct:**
Base case is OUTPUT (ways/value), not INPUT
`if (n==0) return 1` = 1 way to make 0

Mistake 3: 1D vs 2D State

× Using 1D memo for 2D state:
`memo[i]` for knapsack (needs i AND w)

✓ Match memo dimensions to state:
Coin Change: `memo[amount]` (1D)
Knapsack: `memo[i][w]` (2D)

Mistake 4: Iteration Direction

× Forward iteration in 0/1 Knapsack 1D:
Allows using same item multiple times

✓ Backward iteration for 0/1 Knapsack:
Ensures each item used at most once

Forward OK for: Coin Change (unbounded)
Backward for: 0/1 Knapsack (bounded)

Key Insights

When to Use Which Approach?

Start with: Recursion (most intuitive)
Optimize with: Memoization (add cache)
Convert to: Bottom-up (remove recursion)
Finally: Space optimize (if possible)

State Space Design

Ask: What info needed to solve subproblem?

Climbing Stairs: Just position → 1D
House Robber: Just index → 1D
Coin Change: Just amount → 1D
Knapsack: Item & capacity → 2D

State dimensions = memo dimensions!

Top-Down vs Bottom-Up

Top-Down (Memoization):
+ More intuitive, mirrors problem
+ Only calculates needed subproblems
− Recursion overhead

Bottom-Up (Tabulation):
+ No recursion, more efficient
+ Better cache locality
− Calculates all subproblems
− Harder to write initially

Bounded vs Unbounded

Unbounded (can reuse):
- Coin Change
- Forward iteration in 1D
- Stay at same item level

Bounded (use once):
- 0/1 Knapsack
- Backward iteration in 1D
- Move to next item level

Quick Reference Tables

Problem Comparison

Problem	State	Operation	Type
Climbing Stairs	1D (n)	SUM	Count ways
House Robber	1D (i)	MAX	Optimization
Coin Change	1D (amt)	MIN	Optimization
0/1 Knapsack	2D (i,w)	MAX	Optimization

Complexity Summary

Approach	Time	Space
Pure Recursion	$O(2^n)$	$O(n)$
Memoization	$O(n)$	$O(n)$ + stack
Bottom-Up	$O(n)$	$O(n)$
Space-Optimized	$O(n)$	$O(1)$

Space Optimization Possibility

Problem	Possible?	Reason
Climbing Stairs	✓	Fixed look-back (1,2)
House Robber	✓	Fixed look-back (1,2)
Coin Change	×	Variable look-back
0/1 Knapsack	✓	Only need prev row

Learning Progression

Recommended Order

1. **Recursion Basics**
   - Understanding call stack
   - Base case + recursive case
   - Tracing execution
2. **Simple 1D DP**
   - Climbing Stairs (Fibonacci pattern)
   - Identify overlapping subproblems
   - Add memoization
3. **Choice Problems (1D)**
   - House Robber (binary choice)
   - Understanding max/min operations
   - Bottom-up conversion
4. **Unbounded Problems**
   - Coin Change (can reuse)
   - Multiple choices per state
   - Why no space optimization
5. **2D DP**
   - 0/1 Knapsack (bounded)
   - 2D state space
   - Index mapping (i vs i-1)
6. **Space Optimization**
   - When possible (fixed look-back)
   - Backward vs forward iteration
   - 2D to 1D conversion

Pattern Recognition Guide

1D DP with SUM:
→ Counting problems (ways, paths)
Example: Climbing Stairs

1D DP with MAX/MIN:
→ Optimization with constraint
Example: House Robber, Coin Change

2D DP:
→ Two changing parameters
Example: Knapsack (item, weight)

Unbounded:
→ Can reuse elements
→ Forward iteration

Bounded (0/1):
→ Each element used once
→ Backward iteration (for 1D optimization)

Debugging Tips

Common Debugging Steps

1. Check Base Cases
- Are they correct?
- Do they return right type?
- Edge cases covered?

2. Trace Small Example
- Use n=3 or n=4
- Draw recursion tree
- Verify memo updates

3. Verify State
- Does memo match state dimensions?
- Are you checking memo correctly?
- Is memo initialized properly?

4. Check Recurrence
- Is formula correct?
- Are you combining results right?
- Using max/min/sum correctly?

When Answer is Wrong

✓ Print intermediate values
✓ Check memo state after each update
✓ Verify you're returning from right place
✓ Ensure memo is shared (not recreated)
✓ Check array bounds
✓ Verify loop directions (forward/backward)

Final Tips

Problem-Solving Strategy

Step 1: Identify DP
- Optimal substructure?
- Overlapping subproblems?

Step 2: Define State
- What changes between subproblems?
- How many dimensions needed?

Step 3: Write Recurrence
- Express in terms of smaller problems
- Identify base cases

Step 4: Implement
- Start with recursion
- Add memoization
- Convert to iterative
- Optimize space if possible

Interview Tips

- Always start by explaining approach
- Discuss time/space complexity
- Mention optimization possibilities
- Test with small example
- Discuss edge cases
- Know when to stop optimizing

Practice Strategy

Master one problem completely:

1. Pure recursion
2. Add memoization
3. Convert to iterative
4. Space optimize
5. Explain complexity

Then move to next problem.
Don't just memorize solutions!